



Property Valuation — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 23-March-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

1 Test Results — Property Valuation (MKM-PV-001)

Tests run on **23 March 2026 at 11:32:07**.

Table 1: Test Results Summary — Property Valuation (MKM-PV-001)

Total Tests	55
Passed	55
Failed	0
Skipped	0
Duration	18.66s

Table 2: Detailed Test Results — Property Valuation

Test Description	Acceptance Criteria	Result	Time
Line 541: non-dict schema → returns {}.	<code>result == {}</code>	Pass	0.000s
Line 544-545: 'type'/'options'/'description' fields are skipped.	<code>"type" not in result; "options" not in result</code>	Pass	0.000s
Calculate insurance premium flood risk high	<code>premium > 0</code>	Pass	0.096s
Line 214: area=None → computes it first.	<code>isinstance(premium, float); premium > 0</code>	Pass	0.097s
Line 211: property_value=None → computes it first.	<code>isinstance(premium, float); premium > 0</code>	Pass	0.197s
Calculate monthly rent all types	<code>800 <= rent <= 15.000</code>	Pass	0.000s
Lines 165-166: area=None → computes it first.	<code>isinstance(rent, float); rent > 0</code>	Pass	0.000s
Lines 162-163: property_value=None → computes it first.	<code>isinstance(rent, float); rent > 0</code>	Pass	0.101s
Calculate monthly rent positive	<code>isinstance(rent, float); 800 <= rent <= 15.000</code>	Pass	0.000s
Calculate property area all types	<code>area > 0</code>	Pass	0.000s
Lines 128-129: invalid date format → random years_ago.	<code>isinstance(price, float)</code>	Pass	0.000s
Lines 120-121: property_value=None → computes it first.	<code>isinstance(price, float); price > 0</code>	Pass	0.101s
Calculate sale price positive	<code>isinstance(price, float); 50.000 <= price <= 8.000.000</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Lines 135-137: sale_date 1-2 years ago.	<code>isinstance(price, float)</code>	Pass	0.000s
Lines 139-140: sale_date > 2 years ago.	<code>isinstance(price, float)</code>	Pass	0.000s
Lines 124-128: sale_date within 1 year.	<code>isinstance(price, float)</code>	Pass	0.000s
Line 60-61: datetime → isoformat string.	<code>"2024-01-01" in result</code>	Pass	0.000s
Lines 64-65: np.float64 → float.	<code>"3.14" in result</code>	Pass	0.000s
Lines 62-63: np.integer → int.	<code>result == "42"</code>	Pass	0.000s
Lines 66-67: np.ndarray → list.	<code>result == [1, 2, 3]</code>	Pass	0.000s
Line 68: super().default() raises TypeError for unknown type.	—	Pass	0.000s
Lines 510-517: each fallback location has lat/lon/name/elevation.	<code>"lat" in loc; "lon" in loc (+2 more)</code>	Pass	0.000s
Lines 499-518: fallback generates right number of locations.	<code>len(locs) == 5</code>	Pass	0.001s
Lines 505-506: uses params.get_elevation if available.	<code>all(loc["elevation"] == 10.0 for loc in locs)</code>	Pass	0.000s
Lines 608-612: generate_properties returns result dict.	<code>isinstance(result, dict); "data" in result (+1 more)</code>	Pass	0.899s
Lines 610-612: generate_properties creates output file.	<code>result["file_path"].exists()</code>	Pass	0.581s
Calculate mortgage financials returns dict	<code>isinstance(result, dict); "loan_amount" in result (+2 more)</code>	Pass	0.000s
Generate financial data keys	<code>"mortgage_id" in data; "loan_amount" in data (+1 more)</code>	Pass	0.000s
Interest rate range	<code>0.01 <= data["interest_rate"] <= 0.15</code>	Pass	0.000s
Loan amount less than value	<code>data["loan_amount"] <= info["property_value"] * 1.05</code>	Pass	0.000s
Mortgage id format	<code>re.match(r"~MORT-[a-f0-9]{8}\$", data["mortgage_id"])</code>	Pass	0.000s
Quality consistency check	<code>isinstance(corrected, dict)</code>	Pass	0.000s
Calculate insurance premium positive	<code>isinstance(premium, (int, float)); premium > 0</code>	Pass	0.099s
Calculate property area positive	<code>isinstance(area, (int, float)); area > 0</code>	Pass	0.000s
Calculate property value in range	<code>50_000 <= value <= 10_000_000</code>	Pass	0.982s
Generate bedrooms detached	<code>isinstance(beds, int); 1 <= beds <= 7</code>	Pass	0.000s
Generate bedrooms flat	<code>isinstance(beds, int); 1 <= beds <= 4</code>	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Generate postcode format	<code>isinstance(postcode, str); len(postcode) >= 5</code>	Pass	0.000s
Generate field value decimal	<code>isinstance(val, (int, float))</code>	Pass	0.206s
Generate field value text	<code>isinstance(val, str)</code>	Pass	0.099s
Coordinates in thames range	<code>51.0 <= loc["lat"] <= 52.0; -1.0 <= loc["lon"] <= 1.0</code>	Pass	2.972s
Processing stats	<code>stats["successful_properties"] == 5; stats["failed_properties"] == 0</code>	Pass	1.496s
Output file exists	<code>result["file_path"].exists()</code>	Pass	1.468s
Output file is valid json	<code>isinstance(data, dict)</code>	Pass	1.473s
Generate correct count	<code>len(result["data"]["properties"]) == 5</code>	Pass	1.539s
Generate returns expected keys	<code>"data" in result; "file_path" in result (+1 more)</code>	Pass	1.465s
Property id format	<code>re.match(r"~PROP-[a-f0-9]{8}\$", pid)</code>	Pass	1.579s
Property ids unique	<code>len(ids) == len(set(ids))</code>	Pass	3.010s
Construction year in range	<code>1600 <= year <= 2026</code>	Pass	0.000s
Property id format	<code>re.match(r"~PROP-[a-f0-9]{8}\$", meta["property_id"])</code>	Pass	0.096s
Returns dict with required keys	<code>"property_id" in meta; "property_type" in meta</code>	Pass	0.099s
Lines 73-75: generate_grid_reference returns a non-empty string.	<code>isinstance(ref, str); len(ref) > 4</code>	Pass	0.000s
Lines 50-52: generate_owner_name returns 'Firstname Lastname'.	<code>len(parts) == 2; all(p[0].isupper() for p in parts)</code>	Pass	0.000s
Lines 43-45: generate_past_date returns YYYY-MM-DD string.	<code>isinstance(result, str); len(parts) == 3</code>	Pass	0.000s
Custom days_range is respected (result is a past date).	<code>dt < datetime.now()</code>	Pass	0.000s

1.1 Test Document History

Date	Version	Description	Author
05-Mar-2026	1.0	Initial test suite (28 tests)	D. Kelly
08-Mar-2026	1.1	25 test(s) added; 28 test(s) removed (total: 25)	D. Kelly
08-Mar-2026	1.2	4 test(s) added (total: 29)	D. Kelly
09-Mar-2026	1.3	12 test(s) added (total: 41)	D. Kelly
23-Mar-2026	1.4	14 test(s) added (total: 55)	D. Kelly